AI-POWERED SOFTWARE MODERNIZATION

LEGACYBRIDGE

Understand Legacy. Build What's Next.

AI-powered legacy code intelligence, microservices migration planning, and business-logic validation.

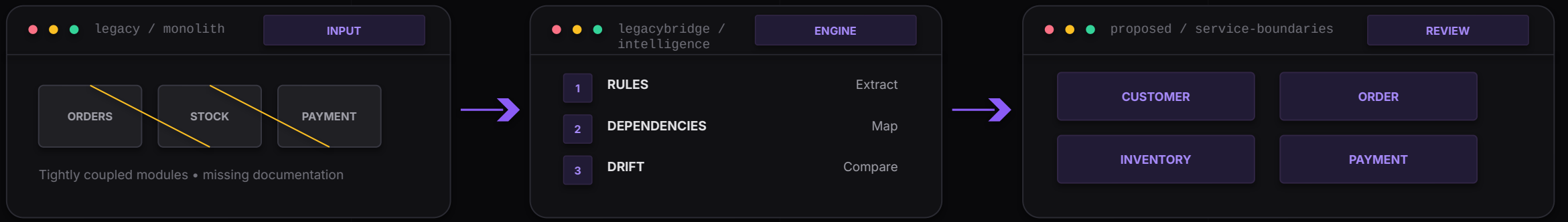
workspace / modernization-brief — PROPOSED

TRANSFORMATION BRIEF

Recover the rules.
Map the dependencies.
Validate the behavior.

DISCOVER | PLAN | VALIDATE

A CONNECTED MODERNIZATION SYSTEM

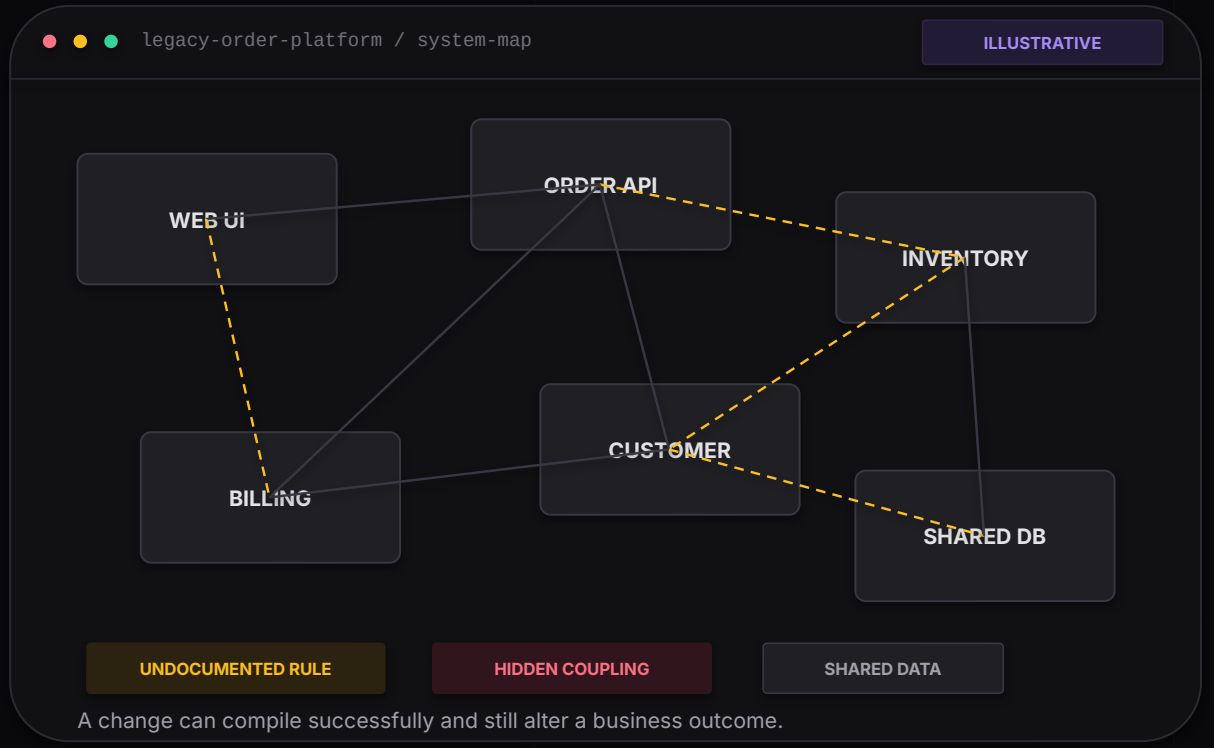


TEAM	MEMBERS	HACKATHON	INSTITUTION
[Team name]	[Names]	[Event name]	[Institution]

THE PROBLEM

Modernizing Legacy Software Shouldn't Require Guesswork.

The hardest risks are often invisible before the first code change.



A change can compile successfully and still alter a business outcome.

01 Undocumented business logic

Rules live in code and institutional memory, not in usable documentation.

02 Hidden dependencies

Unclear module and data relationships make every change harder to reason about.

03 Unclear service boundaries

Splitting a monolith without business and data ownership creates new coupling.

04 Behavioral regression risk

A migrated implementation can build and deploy while changing important decisions.

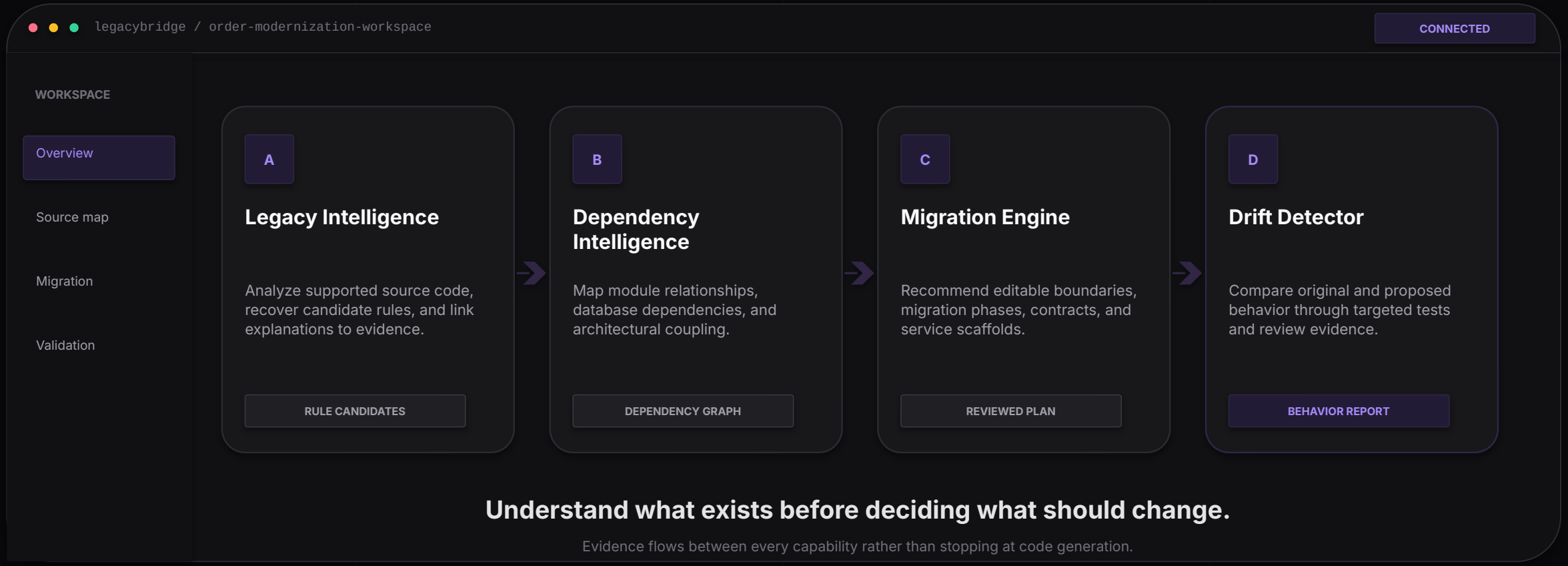
CONNECTED CONSEQUENCES

MANUAL INVESTIGATION	expensive
SLOW MODERNIZATION	delayed
REGRESSION RISK	behavioral
KNOWLEDGE TRANSFER	difficult

THE LEGACYBRIDGE SOLUTION

From Legacy Complexity to Evidence-Based Modernization.

One connected workspace for understanding, planning, generating, and validating.



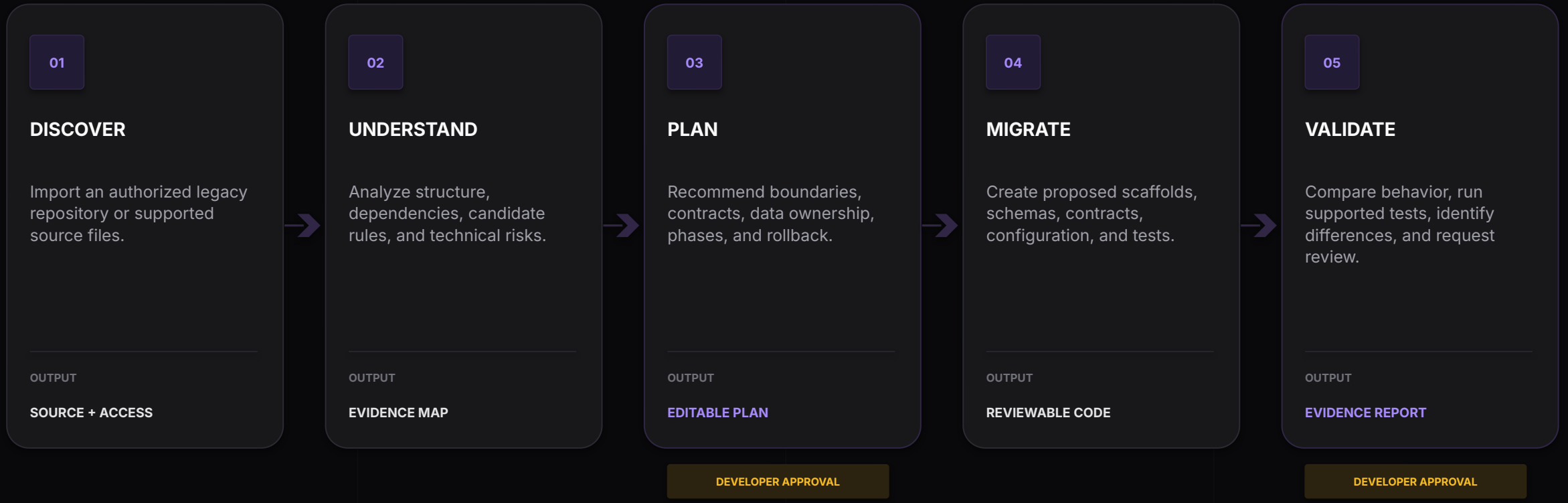
Understand what exists before deciding what should change.

Evidence flows between every capability rather than stopping at code generation.

HOW IT WORKS

One Connected Workflow. From Codebase to Validated Migration.

DISCOVER → UNDERSTAND → PLAN → MIGRATE → VALIDATE



! Human checkpoints: approve the migration plan before generation, and approve evidence before applying migration changes.

FLAGSHIP DIFFERENTIATOR • BUSINESS LOGIC DRIFT DETECTOR

Catch Business-Logic Changes Before Trusting the Migration.

Illustrative order-management example • original rule versus proposed implementation

legacy / orders / confirm.ts — ORIGINAL

ORIGINAL BUSINESS RULE

```
if (inventory < order.quantity) {
  return reject("INSUFFICIENT_STOCK");
}
return confirm(order);
```

proposed / order-service / confirm.ts — PROPOSED

MIGRATED DECISION PATH

```
// inventory guard omitted
return confirm(order);

// builds successfully
```

DECISION CONDITION CHANGED

GENERATED REGRESSION TEST • ILLUSTRATIVE EVIDENCE

TEST INPUT	EXPECTED	OBSERVED	REVIEW STATUS	NEXT ACTION
inventory: 2 order.quantity: 5	**REJECTED** **INSUFFICIENT_STOCK**	**CONFIRMED** illustrative result	**BLOCK APPROVAL** **unverified in repo**	Restore guard rerun isolated test

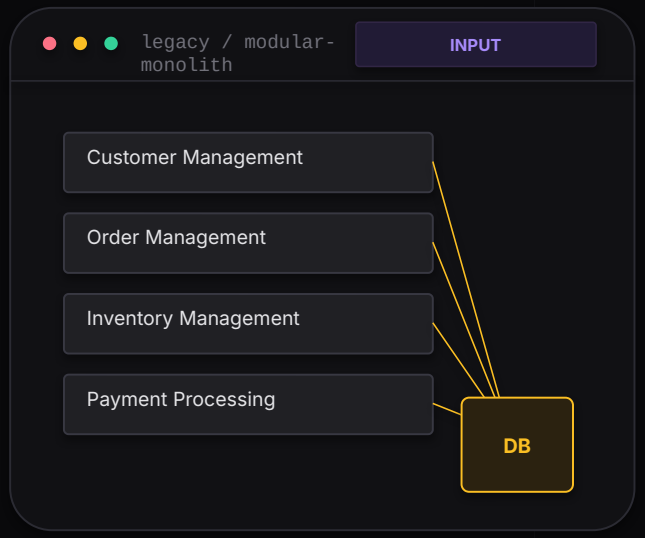
AI-INFERRED OR UNVERIFIED until executed • A passing suite never proves complete behavioral equivalence.

Don't just migrate the code. Verify the behavior.

MONOLITH-TO-MICROSERVICES MIGRATION ENGINE

Turn a Monolith into a Reviewed, Incremental Migration Plan.

Candidate boundaries are recommendations to inspect—not universal architectural answers.



PROPOSED BOUNDARIES • EDITABLE

C

Customer Service

Owns customer data

API + DATA

O

Order Service

Owns order workflow

API + DATA

I

Inventory Service

Owns stock state

API + DATA

P

Payment Service

Owns payment records

API + DATA

Dependency-aware sequence: extract the lowest-risk boundary first, preserve rollback, and review every phase.

MIGRATION CAPABILITIES

- Boundary evidence
- API + event contracts
- Data ownership
- Incremental phases
- Starter code + tests
- Rollback planning

ARCHITECTURE CHECKPOINT

A modular monolith may remain the better intermediate architecture when an immediate split adds unnecessary complexity.

Migration is not just splitting code into folders. It is defining clear business and data boundaries.

TINYFISH MIGRATION RESEARCH AGENT

Ground Migration Decisions in Relevant Technical Documentation.

External research is source-linked and reviewable; source-code analysis remains inside LegacyBridge.

1 LegacyBridge Analysis → 2 TinyFish Research → 3 Official Documentation → 4 Cited Recommendation → 5 Developer Review

ILLUSTRATIVE RESULT • NOT A LIVE QUERY

RECOMMENDATION

Review documented breaking changes for the selected framework version before approving the migration scaffold.

DOCUMENTATION TITLE + URL

Next.js Documentation

`https://nextjs.org/docs • source domain: nextjs.org`

SUPPORTING NOTE Paraphrased: use the official guide relevant to the current and target versions.

Timestamp: [populated at runtime] • Relevance: framework selected by analysis • Limitations: does not establish project compatibility

SEPARATION OF RESPONSIBILITIES

TinyFish

Searches and evaluates external sources, then synthesizes cited research for migration context.

LegacyBridge

Inspects authorized source code, proposes architecture changes, and runs supported tests.

NO AUTOMATIC CONVERSION

SECURITY • Server-side credentials • Never send confidential source code or secrets to the external research agent • Verify source relevance

TECHNICAL ARCHITECTURE AND TECH STACK

An Extensible Architecture Built for Safe Modernization.

Proposed architecture • each technology has a specific responsibility

01	**USER EXPERIENCE** Accessible project workspace and review flows	Next.js • React • TypeScript • Tailwind • Framer Motion
02	**APPLICATION SERVICES** Validation, orchestration, permissions, and jobs	Route Handlers • Zod • migration planner • job coordination
03	**ANALYSIS + INTELLIGENCE** Parse, reason, compare, research, and test	Parsers • OpenRouter • Drift Detector • TinyFish • isolated workers
04	**DATA + IDENTITY** Private projects, identity, and traceable evidence	Clerk • Supabase PostgreSQL • private Storage • RLS • Pinecone optional
05	**DELIVERY + OBSERVABILITY** CI/CD, deployment, monitoring, and test tooling	GitHub Actions • Vercel • Cloudflare • Resend • PostHog • Sentry • Docker • Vitest • Playwright

UNTRUSTED CODE → ISOLATED WORKER • NEVER ORDINARY WEB REQUESTS

SECURITY STRIP

1	**Private source storage**	PROJECT
2	**Server-side secrets**	ENV
3	**Project-level access**	RLS
4	**Isolated execution**	WORKER
5	**Human approvals**	REVIEW
6	**Traceable results**	AUDIT

OPTIONAL

Pinecone for semantic retrieval
Stripe only if payments launch

Docs: nextjs.org/docs • supabase.com/docs • clerk.com/docs • docs.github.com/actions

SECURITY, GOVERNANCE AND LIVE DEMONSTRATION

Trust Through Evidence, Isolation, and Human Oversight.

A credible demo distinguishes working UI, simulated states, planned execution, and unavailable integrations.

SAFEGUARDS BY DESIGN

- ✓ Authorized repository access
- ✓ Private project storage + policies
- ✓ Secret redaction for permitted AI
- ✓ Controlled archive handling
- ✓ Isolated untrusted-code execution
- ✓ Source-linked analysis findings
- ✓ Audit events for key actions
- ✓ Human approval before changes

HACKATHON DEMO SEQUENCE

1	Import sample order-management monolith	UI
2	Discover modules + dependency graph	SIM
3	Review proposed service boundaries	SIM
4	Generate proposed migration scaffold	SIM
5	Introduce deliberate business-rule change	SIM
6	Run regression test + show actual result	PLAN
7	Review TinyFish cited guidance when configured	CONFIG
8	Export validation report	SIM

UI = current interface • SIM = simulated prototype state • PLAN = planned live execution • CONFIG = integration required

VERIFIED HERE: Vite/React UI + Clerk-ready auth | SIMULATED: analysis, graph, plan, drift, research, export | NOT IMPLEMENTED: real parsers, scaffolds, isolated execution, external integrations

IMPACT, ROADMAP AND CLOSING

Make Legacy Modernization More Understandable, Testable, and Controlled.

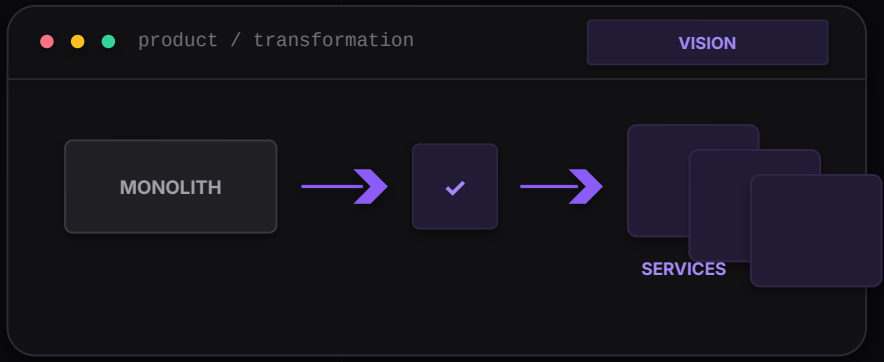
Intended value for teams maintaining business-critical legacy applications.

TARGET USERS

- Modernization teams
- Enterprise developers
- Engineering consultants
- Legacy system owners

INTENDED VALUE

- Less manual dependency investigation
- Visibility into undocumented rules
- Structured migration planning
- Earlier drift detection + traceable decisions



REALISTIC PHASED ROADMAP • CURRENT REPOSITORY IS A UI PROTOTYPE

PHASE 1 — CURRENT FOCUS

WORKING MVP

Code ingestion, source analysis, dependency visualization, and evidence-backed explanations.

PHASE 2 — NEXT

INTELLIGENT MIGRATION

Editable boundaries, migration plans, and service scaffold generation.

PHASE 3 — PLANNED

VALIDATION

Drift detection, regression testing, and migration evidence reports.

PHASE 4 — PLANNED

EXPANSION

Broader languages, enterprise governance, and optional semantic retrieval.

LegacyBridge — Understand Legacy. Build What's Next.

TEAM [Team name] • HACKATHON [Event] • MEMBERS [Names] • INSTITUTION [Institution]